Interview Q&A: explain it simply

Read each answer out loud once. If you can say it in your own words, you're ready.

The 30-second pitch

"The old system checked stock and updated it in two steps inside every request, so under load two people could both buy the last phone, and 100,000 requests fought over one database row. I moved the 'who gets a phone' decision into Redis, using a Lua script that is atomic. The API answers in about a millisecond with 202 Accepted, puts the order on a queue, and workers save it to Postgres with a conditional update. The database never sees the rush, and there are four separate layers that make overselling impossible. I proved it with load tests: with 1,000 users and 20,000 requests the new design sold exactly 500 at about 1,860 requests per second, while the old design sold 3,000 phones out of 500."

Core concepts

Q: What exactly caused 540 sales? A race condition. Request A reads "1 left", request B reads "1 left" before A writes, both write "0", both create an order. This is called a *lost update* or *check-then-act* race.

Q: What is the row-lock bottleneck? Every request updates the same row (product 1). The database lets only one transaction change a row at a time, so 100,000 requests wait in line for one lock and time out.

Q: What's the N+1 problem here? Each request did several separate database queries. With N requests that's N × several round trips. My hot path makes zero database calls; only the ~500 winners ever reach the database.

Q: Why is the Lua script atomic? Redis runs commands one at a time on a single thread. A Lua script runs completely before any other command starts, so check-and-decrement can't be interrupted.

Q: Why not just use `SELECT ... FOR UPDATE` (pessimistic locking)? It's correct, but every request has to wait for the row lock, so with 100k requests most would time out. It's correct but slow.

Q: Optimistic locking? Each update says "only if the version hasn't changed"; if it did, retry. When everyone hits the same row, almost every request conflicts and retries, creating a retry storm.

Q: Then why do you still have `WHERE stock > 0` in the worker? Defense in depth. If Redis were ever wrong (crash, bug, manual edit), the database still refuses to go below zero. There's also a `CHECK (stock >= 0)` constraint and a `UNIQUE (user_id, product_id)`.

Q: Why 202 and not 200? 202 means "accepted for processing, not finished yet". The order is reserved but not saved to the database yet, so we give a tracking URL to check status.

Go questions

Q: Why Go? Goroutines are very cheap (a few KB each), so one server can hold thousands of concurrent requests. The checkout is I/O-bound (waiting on Redis), and Go handles that well.

Q: Goroutine vs thread? A goroutine is a lightweight task managed by the Go runtime, not the OS. Thousands of goroutines run on a few OS threads.

Q: Is your API thread-safe? Yes. The API keeps no shared in-memory state at all: no global maps, no locks. Everything shared lives in Redis, which handles concurrency for us.

Q: Why not cache "sold out" inside each API server to save Redis calls? I considered it, but a user retrying after a network error could be told "sold out" even though they already own a phone. The Lua script checks "does this user already have an order?" first, so going to Redis keeps it correct. Redis handles ~100k of these per second, so the saving wasn't worth the bug.

Q: What does "stateless" mean, and why does it matter? The API keeps nothing in memory between requests. Any replica can handle any request, so the load balancer can send traffic anywhere and we scale by adding more copies.

Queue and workers

Q: Why a queue? It decouples fast from slow. The API accepts orders as fast as Redis allows; workers save them as fast as the database allows. Spikes are absorbed by the queue.

Q: Why Redis Streams and not Kafka/RabbitMQ? Redis is already there, and Streams give consumer groups, acknowledgements, and redelivery. For this scale (500 orders) another system adds cost without benefit. The design works the same with SQS or Kafka.

Q: A worker crashes halfway. What happens? It never acknowledged the message, so it stays "pending". Another worker reclaims it after 30 seconds. If the first worker had already committed, the `order_id` primary key check makes the second attempt a no-op.

Q: What if Redis crashes? Redis writes to disk every second (AOF). On restart, the API reloads stock from Postgres, the source of truth. Even in the worst case, the database constraints prevent overselling. In production I'd use ElastiCache with a replica and automatic failover.

Frontend

Q: Debounce vs throttle? Debounce waits until clicks stop, then fires once. Throttle allows at most one call per time window. For a buy button I used a hard lock: the first click fires, and the rest are ignored until we get an answer.

Q: Polling vs WebSockets vs SSE? WebSockets keep a connection open per user, which is expensive at 100k users and needs sticky sessions. Polling is simple and stateless; I added jitter and backoff so it doesn't spike. SSE is one-way server push over HTTP and would be my next step.

Q: What is optimistic UI? Show the likely result immediately ("You're in the queue") and correct it if the server says otherwise. The app never looks frozen.

Q: How did you keep the bundle small? No framework. One 9.7 KB HTML file with inline CSS, JS, and an SVG image: one request, about 4 KB gzipped.

Q: Why a CDN? Static files are served from edge locations near users, so the API servers only handle dynamic requests.

AWS

Q: What does the ALB do? It gives one public URL, spreads requests across API instances in two Availability Zones, health-checks them every 10 seconds, and stops sending traffic to unhealthy ones.

Q: How is it secured? Only the load balancer is public. API instances accept traffic only from the ALB's security group; Redis and Postgres only from the API security group. Redis has a password.

Q: How would you scale this to 1 million users? Scale the API horizontally (ECS auto scaling, pre-scaled before the sale), ElastiCache with replicas, CloudFront for static files, WAF for bots, and a virtual waiting room in front if needed. The database doesn't need to grow, because only winners reach it.

Likely "gotcha" questions

Q: Can one person buy 2 phones? No. The Lua script stores `userId → orderId`, and the database has `UNIQUE (user_id, product_id)`.

Q: Your rate limiter can be bypassed with a header? Only with a secret key that our own load tester uses (all test traffic comes from one IP). In production that key wouldn't exist, and limits would be enforced by AWS WAF.

Q: What's missing? Payment with a reservation timeout (return unpaid phones to stock after 10 minutes), real login, and a reconciliation job comparing Redis and Postgres.